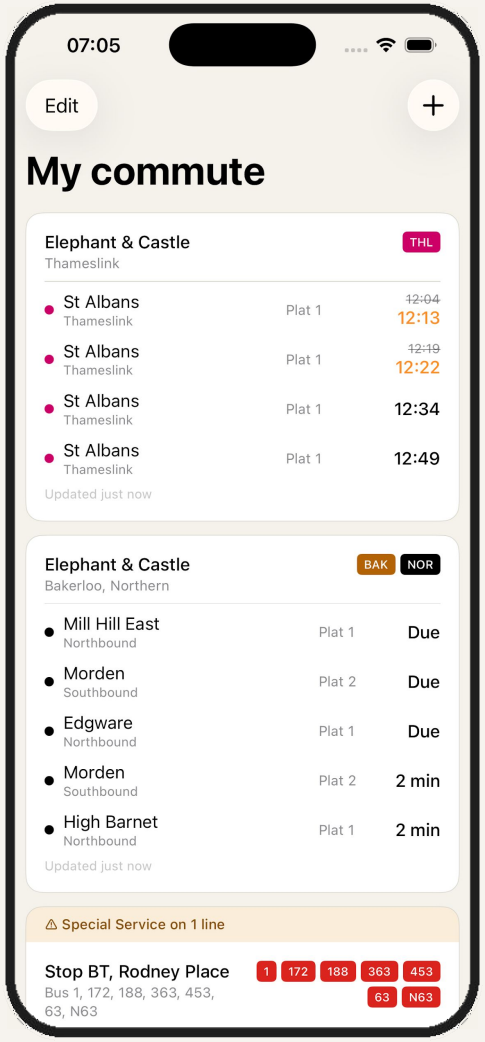



PLATFORM

# Building an iOS app as a non-technical person with agentic coding workflows

# Why I did this

1. Maximise learning velocity through hands-on experimentation
2. Feel the thrill of solving a real problem for myself
3. Develop more informed views on what agentic coding can do today and implications for work cross-sector
4. Be a better commercial person by better understanding the product and technical worlds of colleagues
5. Justify purchases of a mechanical keyboard and various Apple products to my wife

# Evolution of Skillset in 4 Weeks

1

WEEK 1

### Starting from Zero

Only used Claude to answer questions. Didn't know how to open a terminal.

2

WEEK 1

### hello-linda-world

One-shot exercise. Learned GitHub basics and version control.

3

WEEK 2

### built-different-timer

First real build. Learned prompting for Claude Code, project structure, feature branches. Created a reusable Notion template. Wrote down every prompt.

4

WEEK 3

### Autonomous Workflow

Realised CLAUDE.md + BACKLOG.md could replace manual prompting. Claude Code works through items on its own. Practiced on a duplicated repo of my built-different-timer.

5

WEEK 4

### Plf

Applied everything to a more complex build with a public API. Pushed length of autonomous coding sprints to 30 minutes.

# If you're non-technical, limit the blast radius

Assume you'll trigger worst-case scenario.

- Personal laptop, personal Claude account; work tech not involved
- No money tied to API; worst case is hitting rate limits
- No personal information stored in the app
- Claude Code sandboxed; no actions outside the project without approval
- CLAUDE.md guidelines reduce chance of merging without review or printing secrets
- Added complexity incrementally (i.e. first project had no API, this one has a free public API…)


# Project Set Up

# Tools

Claude Chat and Claude Code played different roles; Project files held context & code.

### Claude Chat
*(claude.ai)*

- Thinking partner, builds context through dedicated project
- Refines project brief via Notion connector (read + limited write permissions)
- Designs wireframes
- Researches APIs
- Writes backlog items
- Answers “why” questions

### Project Files
*(shared context)*

- CLAUDE.md (architecture, conventions, guardrails)
- BACKLOG.md (sprint tasks, success criteria)
- Wireframe PNGs
- Logo and brand assets
- XCode Project (Swift code)

### Claude Code
*(terminal agent)*

- Reads CLAUDE.md and BACKLOG.md
- Writes Swift code
- Runs builds and tests
- Commits to Git
- Updates backlog
- Works autonomously through backlog items

**+XCode for code editing and testing** **+ Github for version control** **+ Notion for project and knowledge management**

# Building towards a detailed build plan

**1**
### Project Brief

Drafted in Notion. Claude Chat asked clarifying questions (tech stack? alert strategy? offline behaviour?) then rewrote the brief with answers folded in.

**2**
### Screen List & Wireframes

Claude Chat proposed 5 screens, asked 3 clarifying questions, produced 7 screens. Then designed SVG wireframes for all 7, iterated based on Linda's feedback.

**3**
### Data Model

Claude Chat researched the TfL API (including making live test calls), then designed the data model bridging API responses to app needs.

**4**
### Phased Build Plan

Claude Chat outlined 8 phases (C-J) with specific tasks and success criteria, ready for Claude Code to execute autonomously.

**Giving Claude Chat read and write access to Notion made collaboratively building project files efficient, and ensured Claude had appropriate context in addition to the Claude Chat project where my chat history and select project files were stored.**

# Notion Project

## Plf - TFL App Build

### Overview

Build a customisable transit companion app for London using the TfL Unified API. Deployed locally on iPhone for Londoners who already know their commute and routes. Allows users to display just the information they care about in a clean, card-based UX.

**Project type:** *Native iOS app · Private distribution (TestFlight)*

### Active Phase

`Phase K – Project Files`

Github repo: https://github.com/linlinmarmar/plf

TFL API Key: Artemis @ TFL **User: Profile**

Darwin (National rail) Access Token: OpenLDBWS

### Resources

- Project Brief - TfL
- Screen List
- Data Model

## Project Brief - TfL

*Purpose, core features, and the users*

### Purpose

Build a customisable transit
locally on iPhone for Londor
display just the information

### Tech Stack

- **Platform:** iOS (iPhone
- **Language:** Swift (native
- **Data source:** TfL Unifie
- **Storage:** Local on-devi

### Core Features

#### MVP

- **Home screen** — Custor
style layout). Each card

## Screen List

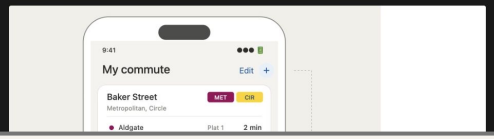
*What each screen does, described without technical jargon*

### Screen 1: Home

*The main screen of the app. A scrollable list of cards — one per saved view — showing real-time arrival information at a glance. If any of the user's saved lines or stations have disruptions, an alert banner appears at the top of the relevant card. When offline, a persistent banner sits above everything indicating data may be stale.*

**What the user can do here:**

- Scroll through their saved view cards, each showing live departure times
- See disruption alerts surfaced on cards for any affected lines or stations
- Tap a card to drill down into full departure details (→ Screen 3)
- Tap an "Add" button (e.g. a `+` in the nav bar) to create a new saved view (→ Screen 4)
- Long-press a card to access a context menu with Edit and Delete options (→ Screen 7)
- Enter edit mode to reorder or remove cards (→ Screen 2)
- See an offline banner when there is no internet connection, with cached data still visible



# The phased build developed with Claude Chat

| | |
|---|---|
| **A** Project brief, screen list, wireframes, data model, branding<br>**B** CLAUDE.md + BACKLOG.md — the control layer for Claude Code | **Project Set Up** |
| **C** Xcode project scaffold, Swift data models, persistence, unit tests<br>**D** TfL API client (5 endpoints), network monitor, error types, tests<br>**E** Navigation shell, empty state screen, visual styling<br>**F** Search with debounced API calls, line/destination selection, save flow<br>**G** Home screen cards with live arrivals, disruption banners, auto-refresh<br>**H** Drill-down view with full departure list, 30-second refresh<br>**I** Edit mode, drag-to-reorder, long-press context menu, edit-existing-view<br>**J** Offline mode, error states, loading skeletons, dark mode, accessibility | **MVP Build** |
| **K** National Rail live departures via Darwin API (biggest post-MVP feature)<br>**L** Bug fixes, official logos, destination filtering fixes, UX polish | **Updates & Fixes** |

*One commit per backlog item. Build must pass with zero errors and zero warnings before moving on. Main always deployable.*

# Documentation before code

The clearer the documentation, the more autonomously Claude Code can work. Ambiguity in the docs leads to guesswork in the code.

**CLAUDE.md (~500 lines)**

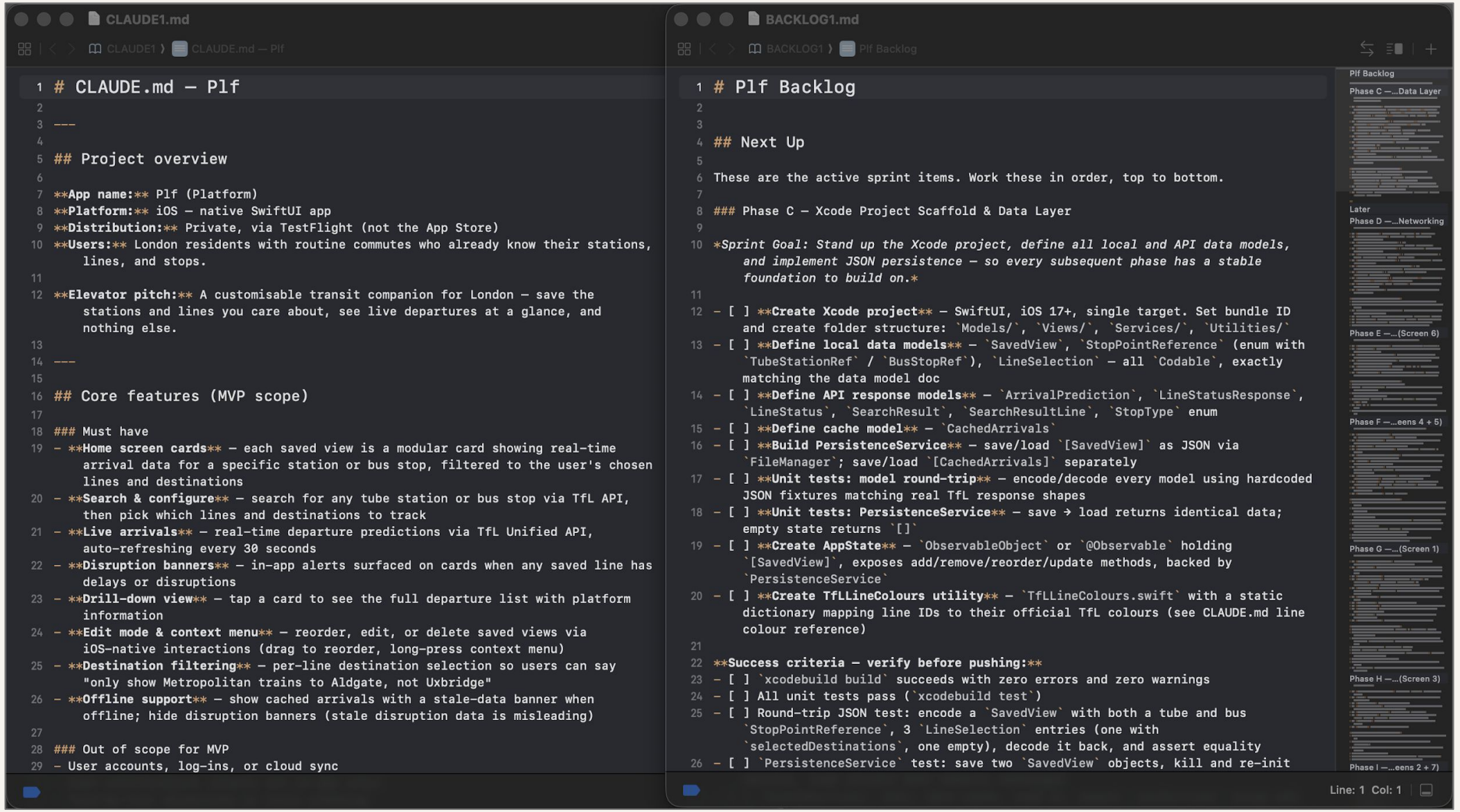
- Project overview and scope
- All 7 screens with interaction specs
- Design language (colours, typography, icons)
- Architecture decisions
- Data model definitions
- API endpoint reference
- Autonomous workflow loop
- Git conventions
- Explicit guardrails

**BACKLOG.md**

- **Next Up** — the active sprint
- **Later** — future work. Do not touch unless explicitly asked.
- **Done** — completed and committed items

*Invest heavily in planning and documentation before writing a single line of code. These two files are the interface between a human product owner and an agent developer. Claude Chat can draft these files with guidance.*

# Documentation before code



CLAUDE1.md

CLAUDE1 › CLAUDE.md — Plf

```
# CLAUDE.md — Plf

---

## Project overview

**App name:** Plf (Platform)
**Platform:** iOS — native SwiftUI app
**Distribution:** Private, via TestFlight (not the App Store)
**Users:** London residents with routine commutes who already know their stations, lines, and stops.

**Elevator pitch:** A customisable transit companion for London — save the stations and lines you care about, see live departures at a glance, and nothing else.

---

## Core features (MVP scope)

### Must have
- **Home screen cards** — each saved view is a modular card showing real-time arrival data for a specific station or bus stop, filtered to the user's chosen lines and destinations
- **Search & configure** — search for any tube station or bus stop via TfL API, then pick which lines and destinations to track
- **Live arrivals** — real-time departure predictions via TfL Unified API, auto-refreshing every 30 seconds
- **Disruption banners** — in-app alerts surfaced on cards when any saved line has delays or disruptions
- **Drill-down view** — tap a card to see the full departure list with platform information
- **Edit mode & context menu** — reorder, edit, or delete saved views via iOS-native interactions (drag to reorder, long-press context menu)
- **Destination filtering** — per-line destination selection so users can say "only show Metropolitan trains to Aldgate, not Uxbridge"
- **Offline support** — show cached arrivals with a stale-data banner when offline; hide disruption banners (stale disruption data is misleading)

### Out of scope for MVP
- User accounts, log-ins, or cloud sync
```

BACKLOG1.md

BACKLOG1 › Plf Backlog

```
# Plf Backlog


## Next Up

These are the active sprint items. Work these in order, top to bottom.

### Phase C — Xcode Project Scaffold & Data Layer

*Sprint Goal: Stand up the Xcode project, define all local and API data models, and implement JSON persistence — so every subsequent phase has a stable foundation to build on.*

- [ ] **Create Xcode project** — SwiftUI, iOS 17+, single target. Set bundle ID and create folder structure: `Models/`, `Views/`, `Services/`, `Utilities/`
- [ ] **Define local data models** — `SavedView`, `StopPointReference` (enum with `TubeStationRef` / `BusStopRef`), `LineSelection` — all `Codable`, exactly matching the data model doc
- [ ] **Define API response models** — `ArrivalPrediction`, `LineStatusResponse`, `LineStatus`, `SearchResult`, `SearchResultLine`, `StopType` enum
- [ ] **Define cache model** — `CachedArrivals`
- [ ] **Build PersistenceService** — save/load `[SavedView]` as JSON via `FileManager`; save/load `[CachedArrivals]` separately
- [ ] **Unit tests: model round-trip** — encode/decode every model using hardcoded JSON fixtures matching real TfL response shapes
- [ ] **Unit tests: PersistenceService** — save → load returns identical data; empty state returns `[]`
- [ ] **Create AppState** — `ObservableObject` or `@Observable` holding `[SavedView]`, exposes add/remove/reorder/update methods, backed by `PersistenceService`
- [ ] **Create TfLLineColours utility** — `TfLLineColours.swift` with a static dictionary mapping line IDs to their official TfL colours (see CLAUDE.md line colour reference)

**Success criteria — verify before pushing:**
- [ ] `xcodebuild build` succeeds with zero errors and zero warnings
- [ ] All unit tests pass (`xcodebuild test`)
- [ ] Round-trip JSON test: encode a `SavedView` with both a tube and bus `StopPointReference`, 3 `LineSelection` entries (one with `selectedDestinations`, one empty), decode it back, and assert equality
- [ ] `PersistenceService` test: save two `SavedView` objects, kill and re-init
```

Plf Backlog
Phase C —…Data Layer
Later
Phase D —…Networking
Phase E —…(Screen 6)
Phase F —…eens 4 + 5)
Phase G —…(Screen 1)
Phase H —…(Screen 3)
Phase I —…eens 2 + 7)

Line: 1 Col: 1


# Core Flow

# How a backlog item becomes code

*Example: National Rail live departures*

### 1. Discovery

Linda tests on device → Elephant & Castle Thameslink shows “Live arrivals not available” → Notices it works for tube not National Rail

### 2. Diagnosis (Claude Chat)

Claude researches and explains: TfL arrivals API doesn’t return predictions for most National Rail operators. This is an API limitation, not a code bug.

### 3. Solution design (Claude Chat)

Claude recommends integrating the Darwin API via Huxley 2 (JSON REST proxy). Writes a detailed build plan with 20+ backlog items.

### 4. Build (Claude Code)

Claude Code works through phase autonomously: new API client, response models, NaPTAN-to-CRS station code mapping, mixed-mode fetch logic, unit tests. Commits one item at a time.

### 5. Test and fix (repeat)

Linda tests → finds destination filtering only matches terminus → writes bug description → Claude Chat formats as backlog item → Claude Code fixes using Darwin’s calling points data

# What Claude Code actually does

For each backlog item, Claude Code follows this autonomous loop:

**1. Read the backlog**
Open BACKLOG.md, find the next unchecked item in “Next Up”

**2. Plan before coding**
State which files to modify, what the change involves, flag risks

**3. Implement the change**
Write the smallest set of changes needed, follow existing patterns

**4. Build and test**
Must pass with zero errors, zero warnings. Run unit tests.

**5. Commit**
One Git commit per backlog item: feat: <short description>

**6. Update backlog**
Mark item done, move to Done section, commit separately

Process and rules outlined in **CLAUDE.md** include: One item at a time. Do not touch “Later” items. Do not refactor unrelated code. If the build breaks, fix it before moving on. If unsure, stop and ask.

```
# CLAUDE.md — Plf
## Current build phase

## Autonomous workflow

When working through backlog items, follow this loop for **each item** in the Next Up section:

### 1. Read the backlog
Open `BACKLOG.md` and identify the **next unchecked item** in "Next Up".

### 2. Plan before coding
State which files you'll modify, what the change involves, and any risks. If anything is ambiguous, **stop and ask** — do not guess.

### 3. Implement the change
- **Always consult the wireframes** in `wireframes/` before building or modifying any screen
- Make the smallest set of changes needed
- Follow existing patterns — match the style of surrounding code
- Use `TfLLineColours.swift` for line colours, `Color.warmBackground` for background, semantic colours for adaptive elements

### 4. Build and test
- **Build:** `xcodebuild build -project plf/plf.xcodeproj -target plf -sdk iphonesimulator26.4 -arch arm64 -quiet CODE_SIGN_IDENTITY=- CODE_SIGNING_REQUIRED=NO CODE_SIGNING_ALLOWED=NO`
- **Test:** `xcodebuild test -project plf/plf.xcodeproj -scheme plf -destination 'platform=iOS Simulator,name=iPhone 17 Pro,OS=26.4' -only-testing:plfTests CODE_SIGN_IDENTITY=- CODE_SIGNING_REQUIRED=NO CODE_SIGNING_ALLOWED=NO`
- Build must pass with zero errors and zero warnings
- If visual, describe what to check on device for TestFlight verification
- Run relevant unit tests and confirm they pass

### 5. Commit and update backlog
- One Git commit per backlog item: `feat: <short description>`
- Do not bundle multiple items into one commit
- Mark the item done in `BACKLOG.md` and move to Done section, then commit separately: `chore: mark "<item>" complete in BACKLOG.md`

### 6. Move to the next item
Repeat until all "Next Up" items are complete, then stop and report.
```

**Autonomous workflow instructions in CLAUDE.md**

**Drafted with support from Claude Chat**

# Have Claude keep documentation up to date:

```
Read CLAUDE.md and BACKLOG.md in the project root. CLAUDE.md contains your
full project context and an "Autonomous workflow" section that describes ex
actly how to process backlog items. Follow that workflow now — work throug
h every unchecked item in the "Next Up" section of BACKLOG.md, one at a tim
e, in order from top to bottom. For each item: plan the change, implement i
t, build to confirm zero errors, commit, then mark it done in BACKLOG.md an
d commit that update. Stop and report when all Next Up items are complete.
Start this sprint with a new feature branch.
```

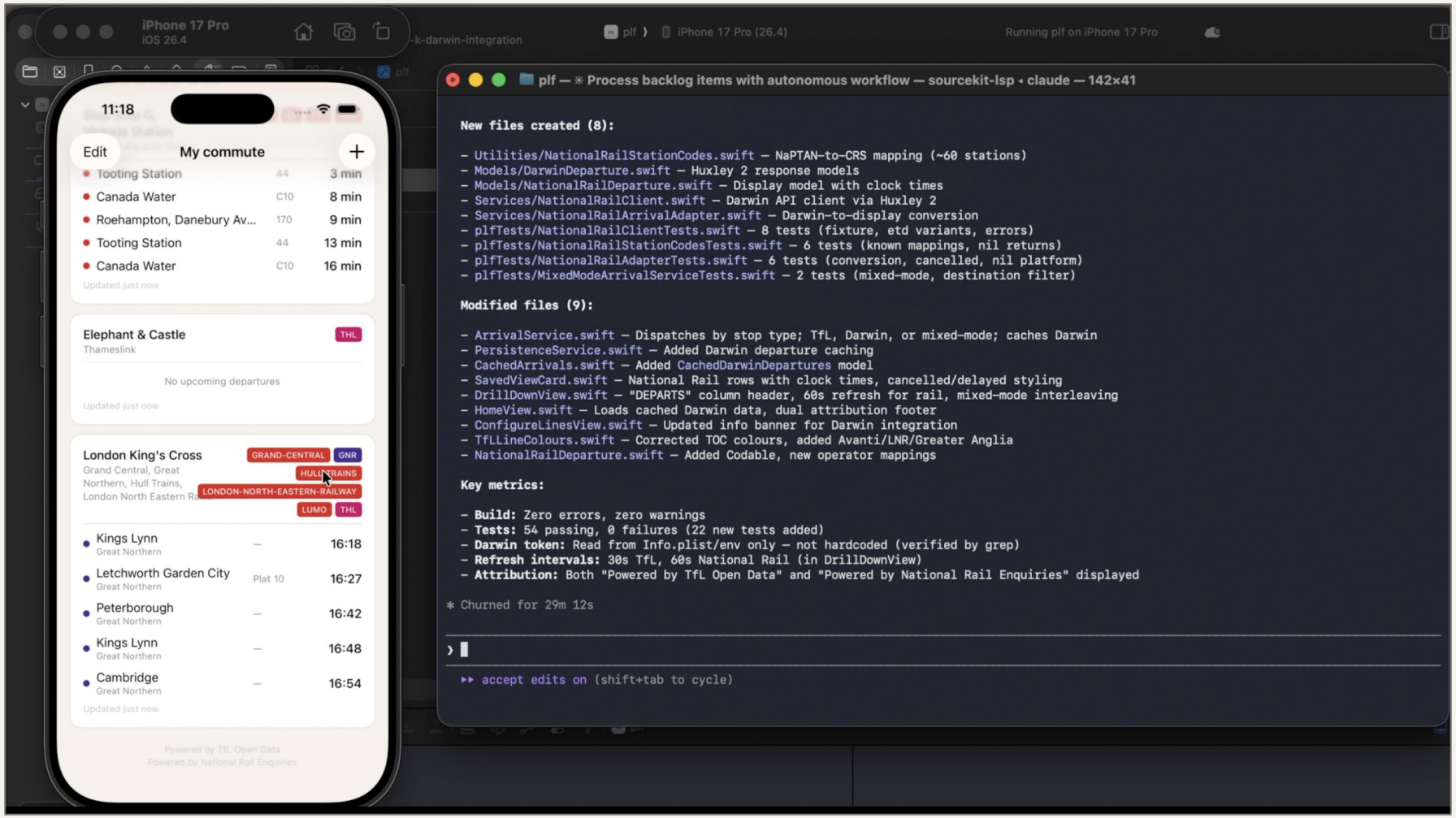
Prompt for start of each phase

```
Update CLAUDE.md with any changes or context that should be carried forwar
d to future phases. Then prepare BACKLOG.md for the next phase. Then commi
t and push to the current feature branch.
```

Prompt for end of each phase

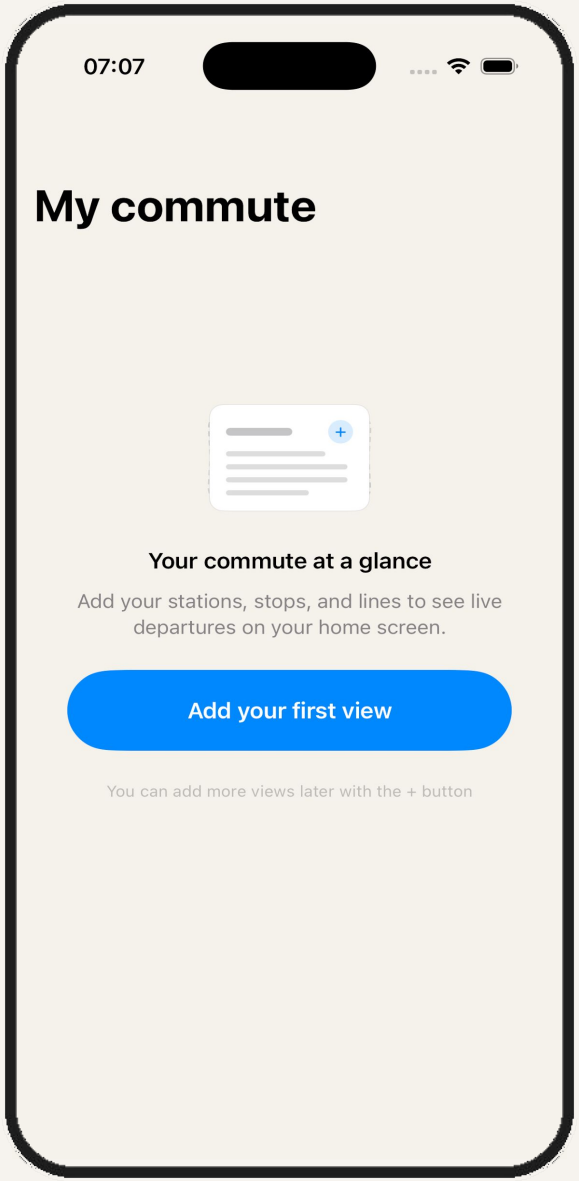
# Phase K ran successfully for nearly 30min straight!

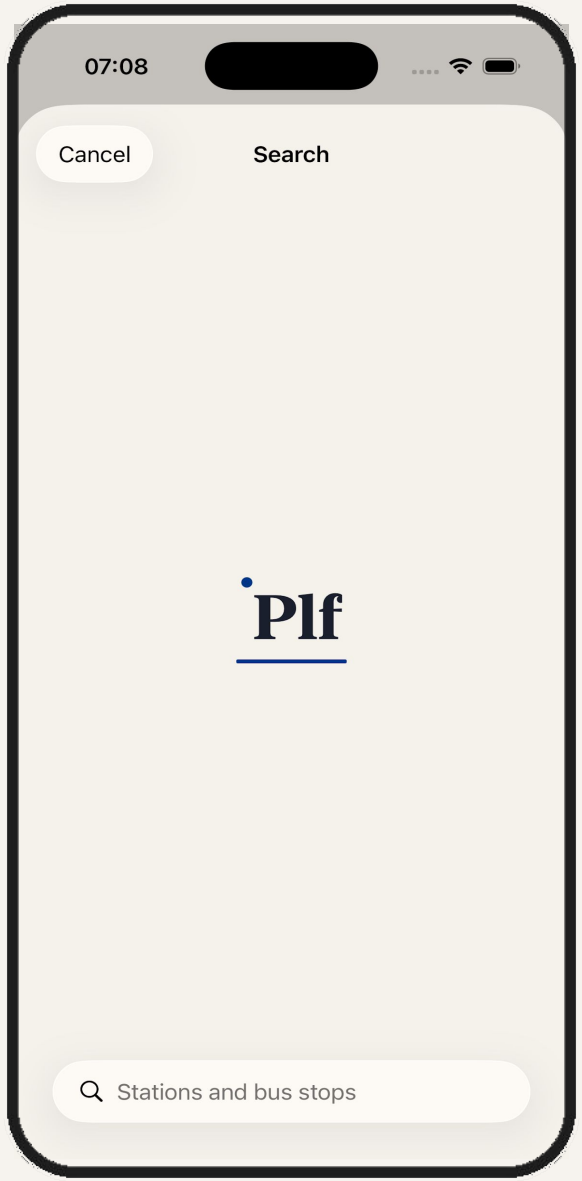
```
* Churned for 29m 12s
```

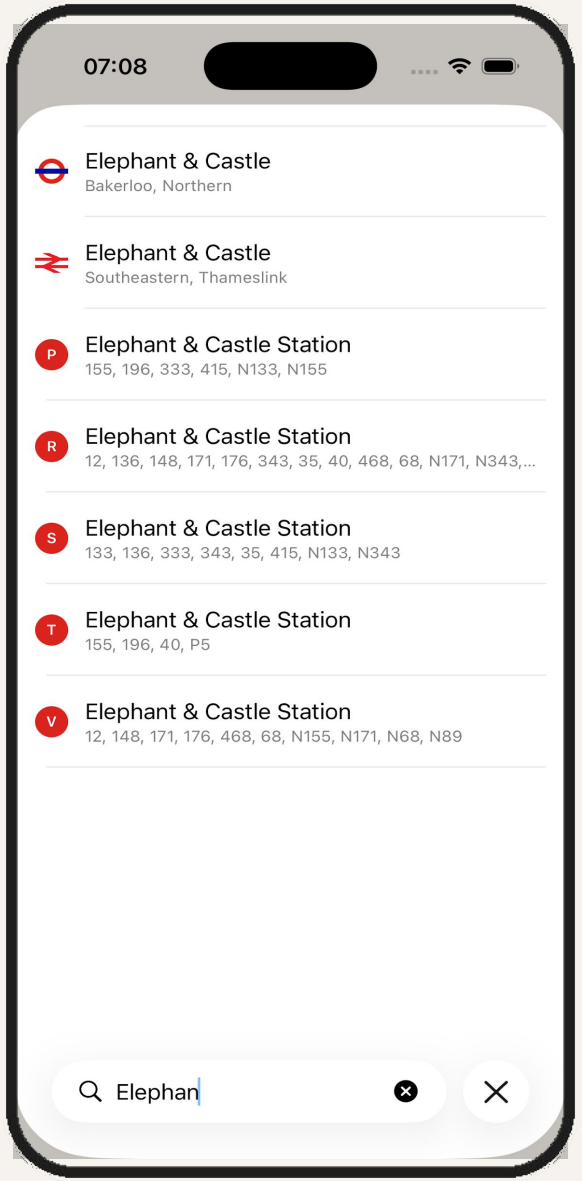


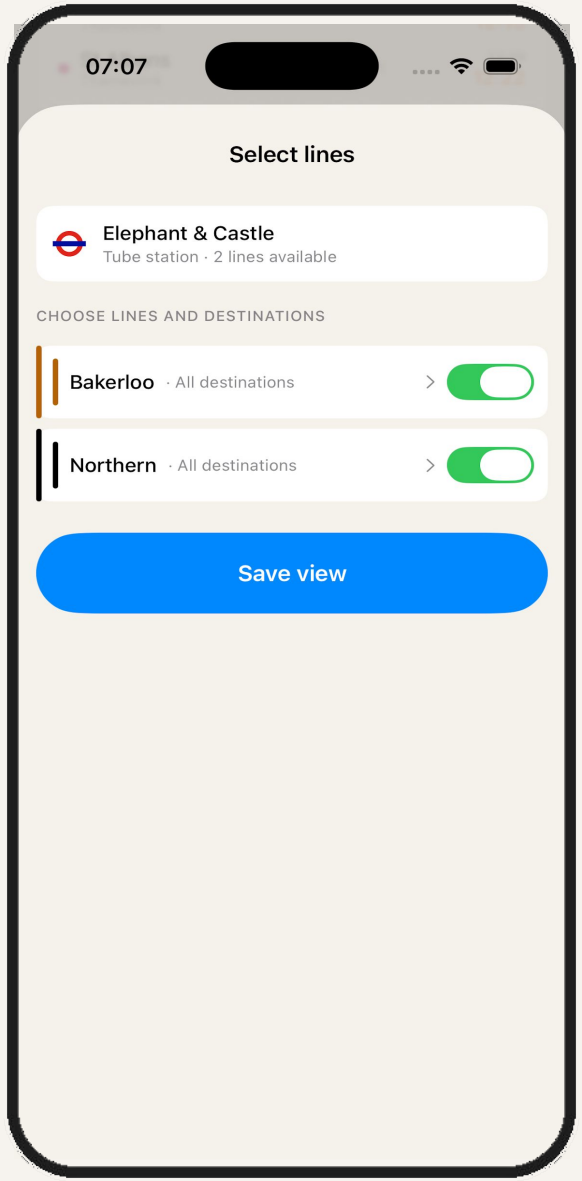

# Screens

# Set up – Add custom views

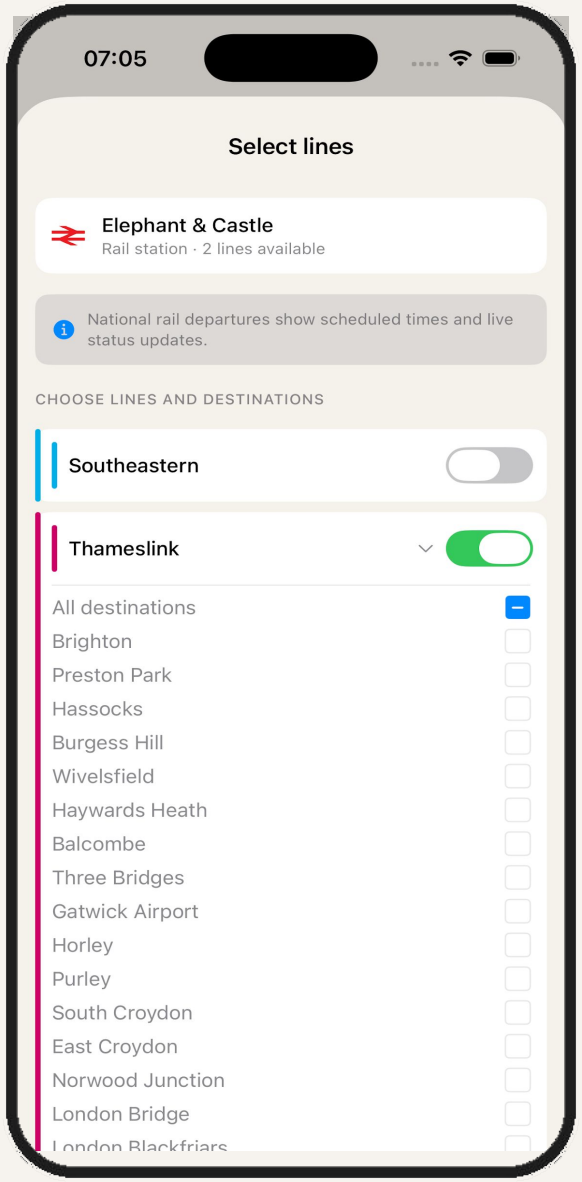


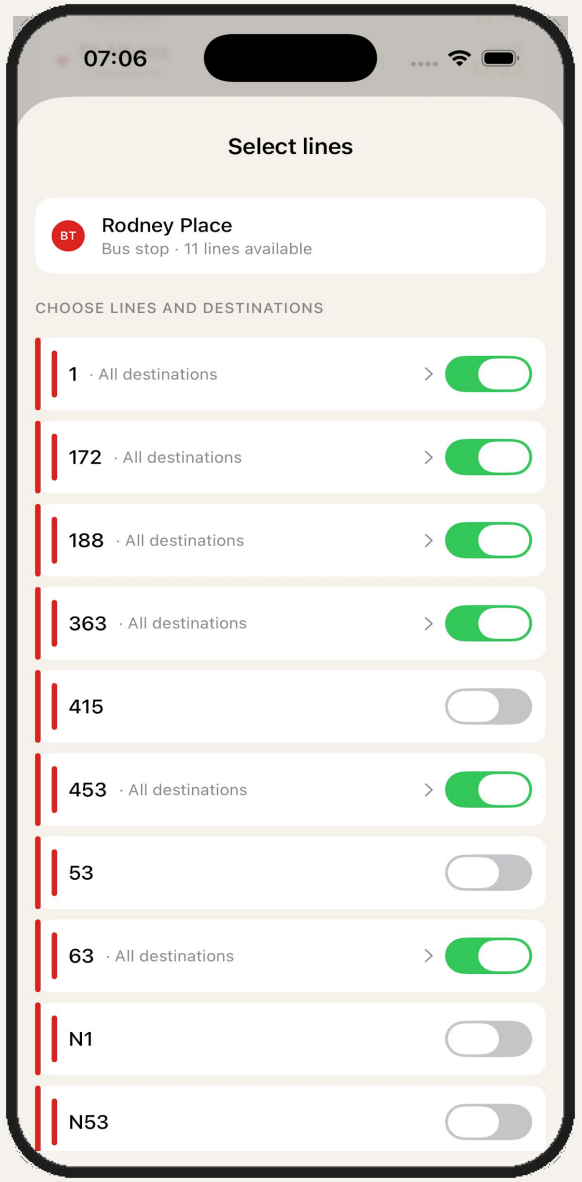



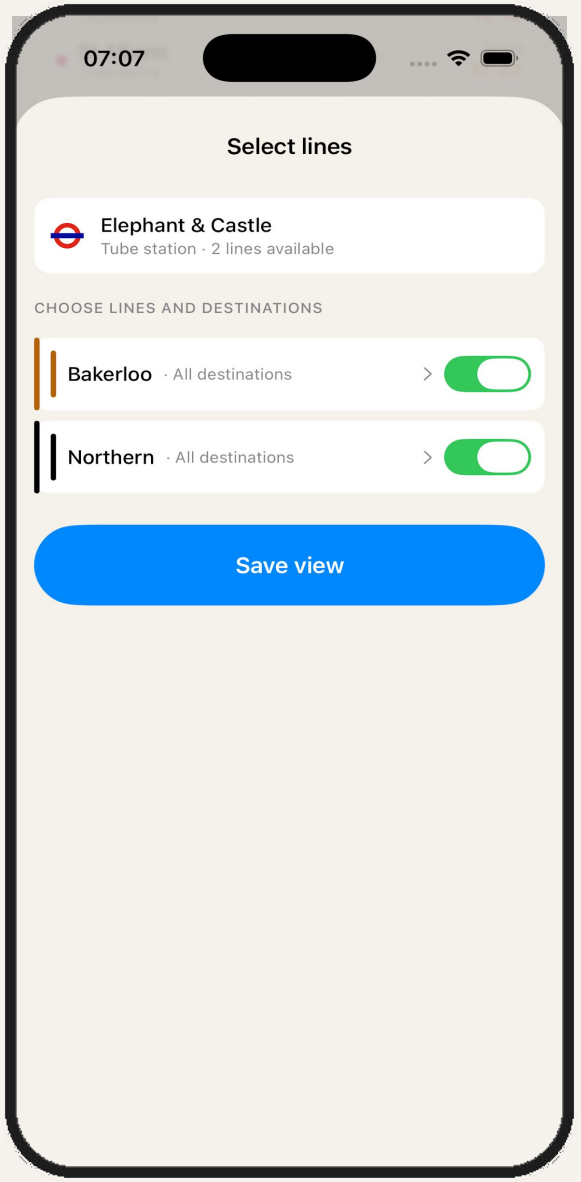



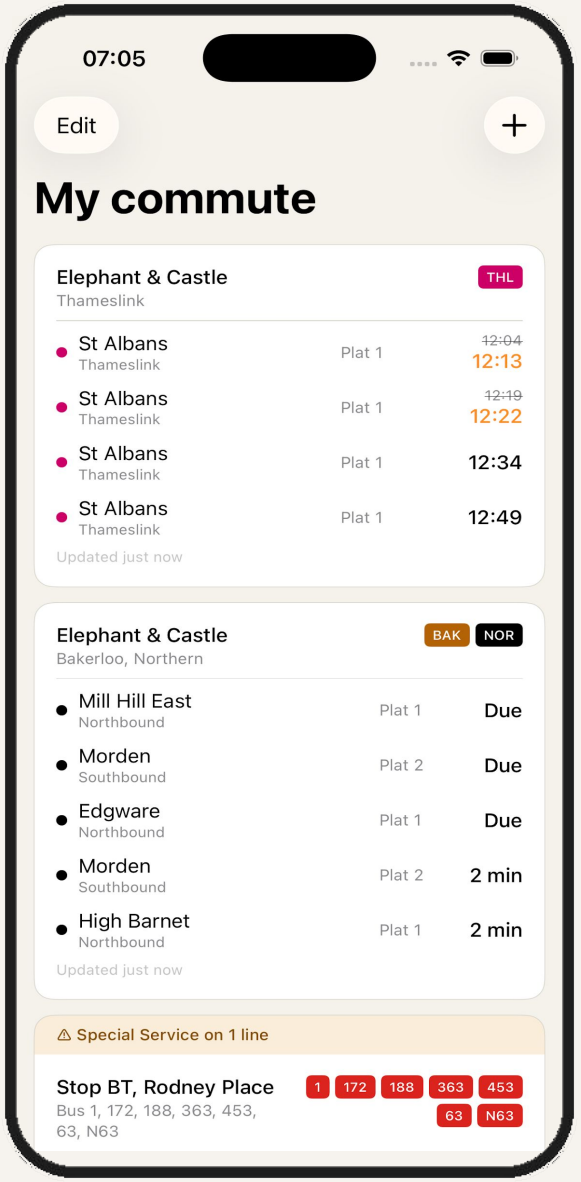



# Add custom tube, rail, and bus views

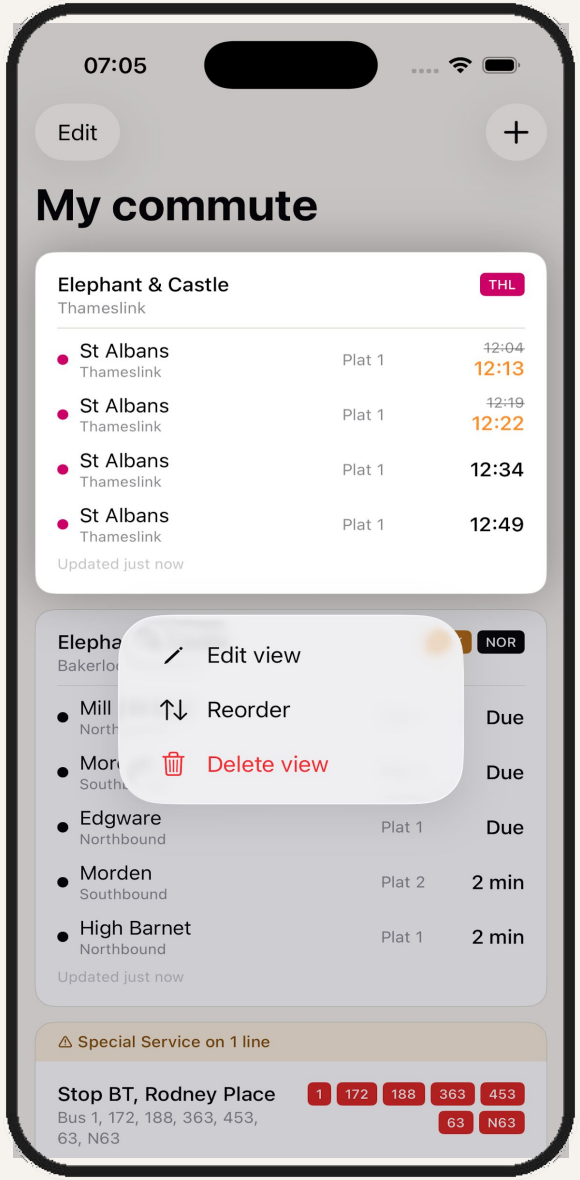


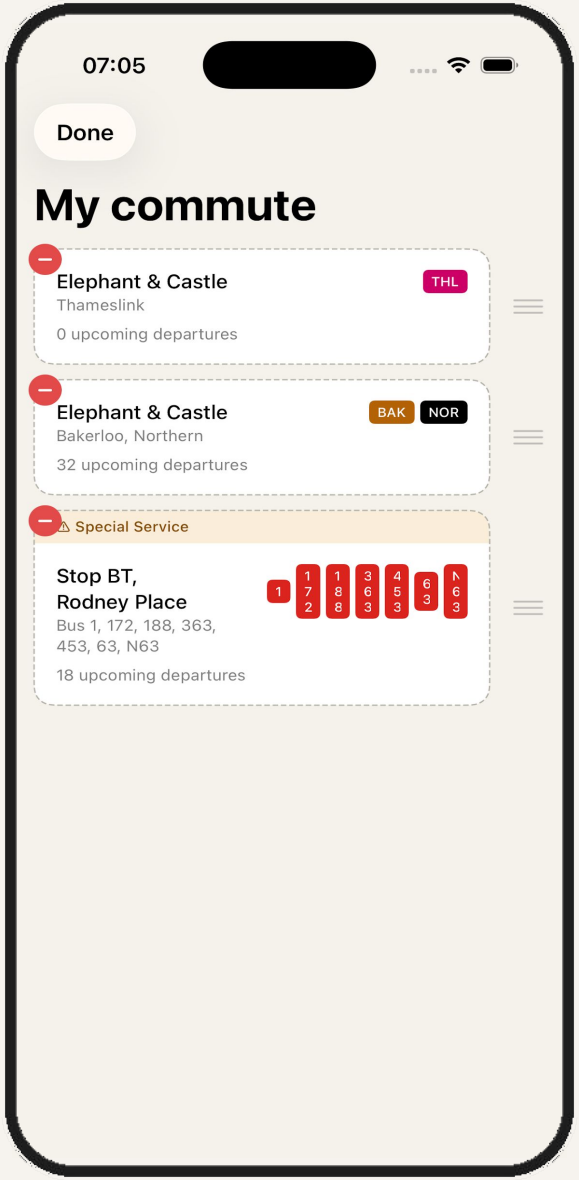



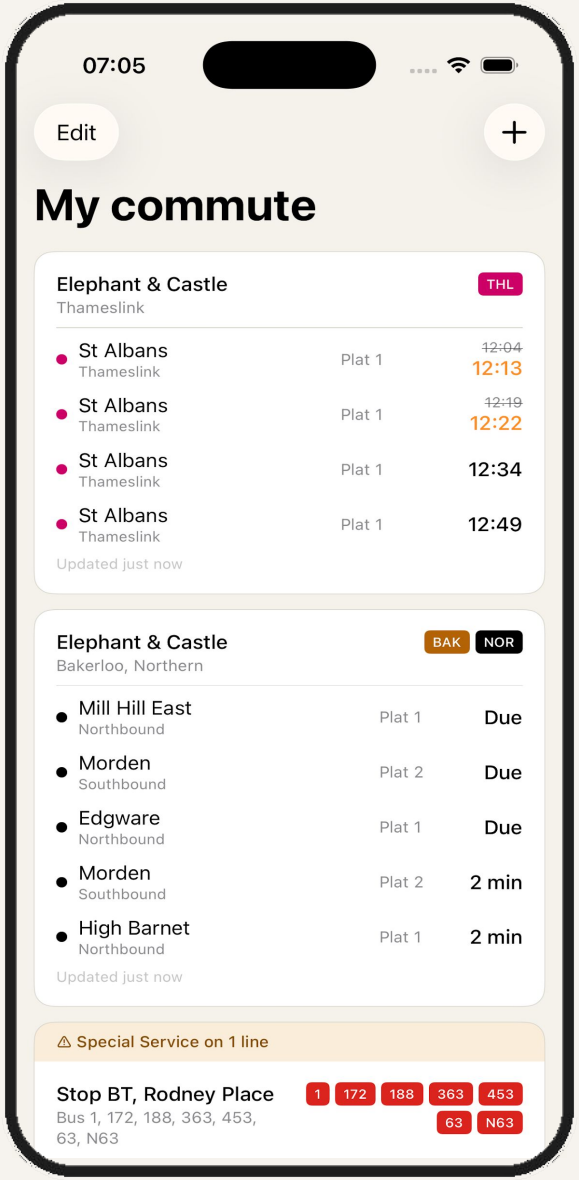



# Edit, reorder, and delete views




# Appendix: On-going Iteration

# Phase N - Major Feature Update

I want to make a major feature update. Right now the title of the home screen says "My commute". Instead, I want to be able to create multiple screens each containing customised views of my choice. For example, I want to be able to easily toggle between "Work" and "Home" and other screens I choose the name of. Help me draft wireframes of these screens. First ask what clarifying questions you need beofre we move onto the wireframes.

**For major feature updates, I use a workflow closer to my Project Set Up. I have Claude Chat (grounded in project context) ask clarifying questions before producing new screens and other artefacts.**

**These artefacts can then be added to the Project Files. They are also a way to validate that Claude Chat is aligned to my vision is before producing instructions (and backlog items) for Claude Code.**

Assessed current architecture before soliciting clarifying questions >

Let me pull up the current project docs to ground my questions in what's already built.

Assessed architecture and identified clarifying questions needed >

Great, I've reviewed the current architecture. This is a significant structural change — moving from a single flat list of `SavedView`s to a grouped, multi-screen model. Here are the questions I'd want to nail down before wireframing:

**1. Navigation between screens** How do you imagine switching between "Work", "Home", etc.? A few options come to mind: a horizontally swipeable page (like iOS Weather), a segmented control / pill bar at the top, or a tab bar at the bottom. Do you have a preference, or would you like me to wireframe a couple of approaches?

**2. Default behaviour & migration** When you upgrade, should the existing "My commute" views all land in a single default screen (e.g. "Commute")? Or would you want a first-run flow that asks you to organise them?

**3. Creating & managing screens** Should there be a dedicated "manage screens" view, or do you picture it more lightweight — e.g. long-press the screen title to rename, a "+" to add a new screen, swipe-to-delete an empty screen? And is there a limit on how many screens you'd want to support?

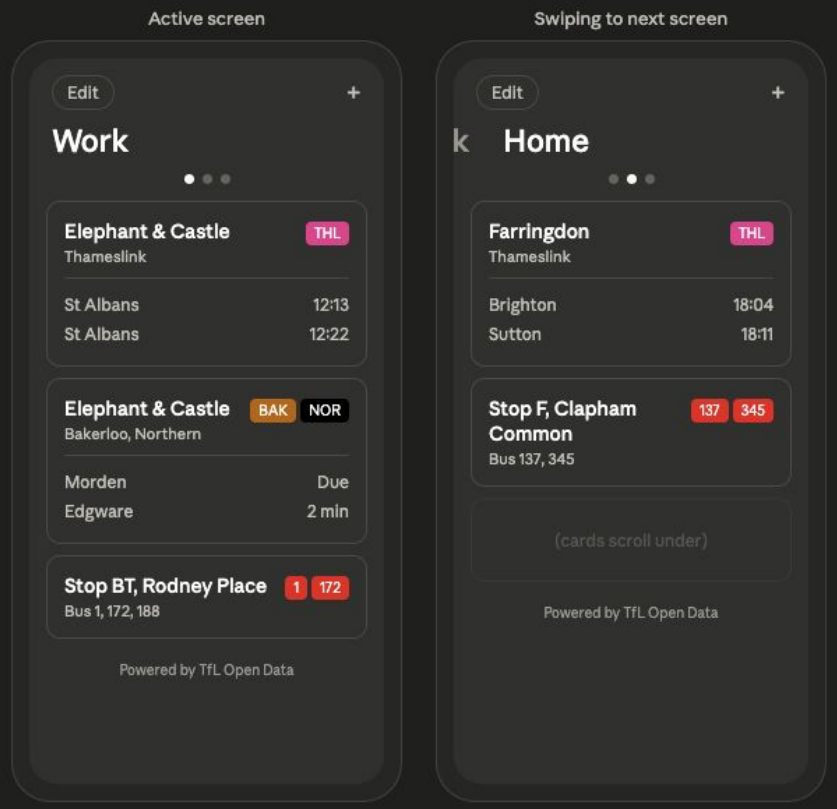
**4. Views belonging to multiple screens** Can a single SavedView (e.g. "Elephant & Castle — Bakerloo") appear on more than one screen? Or is each view exclusive to one screen?

**5. The "Save view" flow** When you add a new view via search → configure lines → save, should you be prompted to choose which screen it lands on? Or does it just go to whichever screen you were looking at when you tapped "+"?

**6. Screen-level settings** Should each screen have its own independent edit mode / reorder? (I'd assume yes, but worth confirming.) Anything else per-screen — like a custom icon or colour?

Let me know your thinking on these and I'll start on the wireframes.

Here are two approaches for the screen-switching navigation. Both mockups show the same content — three screens ("Work", "Home", "Commute") — but with different switching mechanisms.



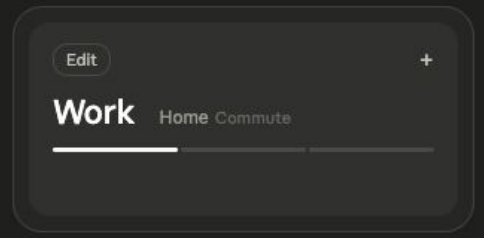

**Option A: swipeable pages**

Swipe left/right to switch screens. Page dots below the title show position. The title cross-fades as you swipe. Like iOS Weather — full-screen gesture, very fluid. Trade-off: you can't jump directly to screen 3 from screen 1.

I don't like the three dots as an indicator for the Option A. Is there a better UI to give this indicator? Brainstorm a couple options.

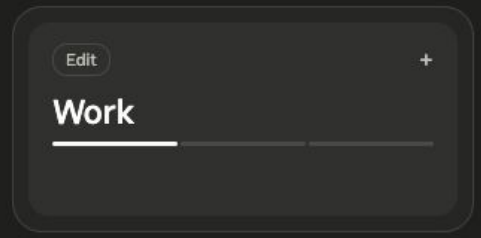
Synthesized multiple UI indicator alternatives for consideration >

Good instinct — dots get noisy fast, especially at 10 screens. Here are four alternatives:



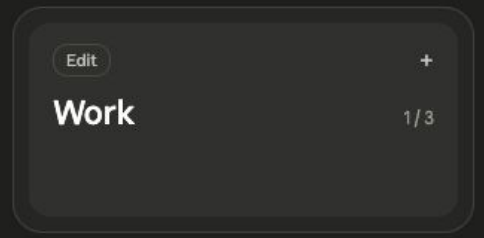

**A. Sliding title strip**

Active name is large, neighbours peek in at smaller size and fade out with distance. A thin segmented bar underneath shows position. The titles themselves scroll as you swipe.




**B. Segmented progress bar**

Just a thin segmented line under the title. One segme
barely
and sta




**C. Counter badge**

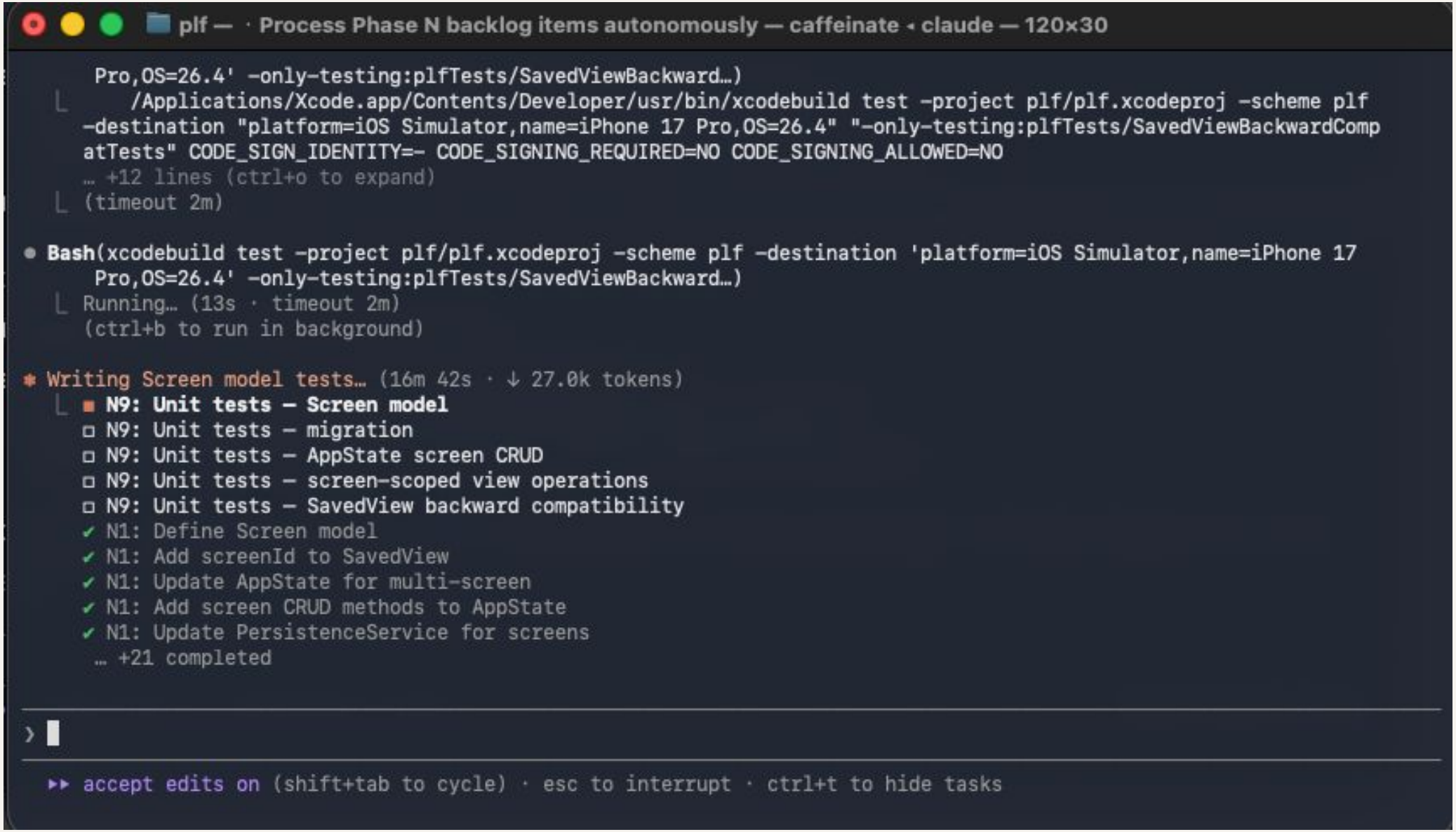
A quiet "1 / 3" sits right-aligned beside the title. Absolute minimum footprint — no extra visual element at all. Always tells you exactly where you are. Might feel too understated for discoverability.

**D. Car**

The ne
hinting there's more to swipe to. Physical and intuitive — you can see something is there. No indicator element needed. Might feel cramped on smaller phones.

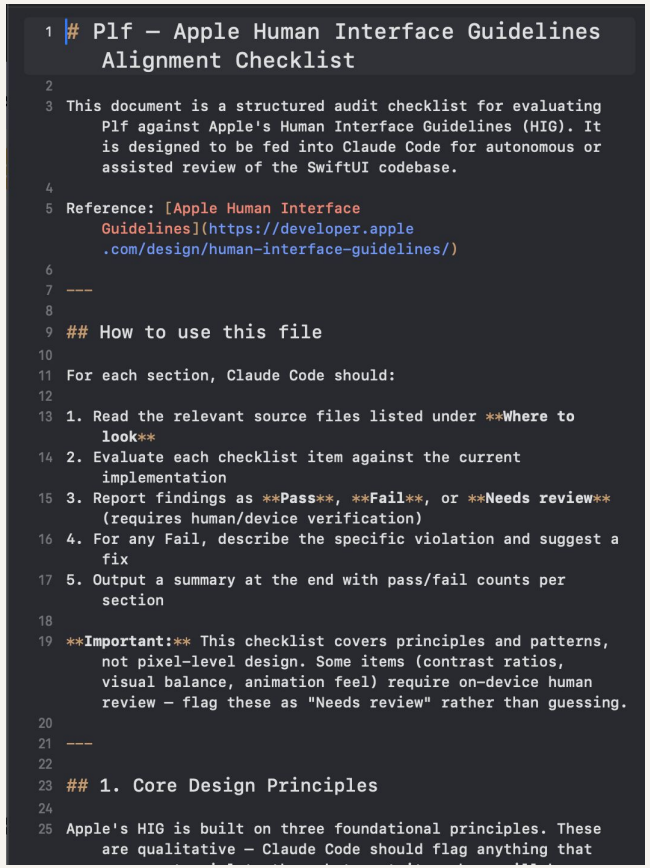
I have added these in the project root folder in a folder called "wireframes-phase-n". Write detailed backlog items for phase n for the multi-screen customisable views feature.

**Quick wireframing let's you quickly explore more optionality for UI/UX**




**Claude Code working through Phase N major feature update**

# Phase P - Apple Human Interface Guidelines

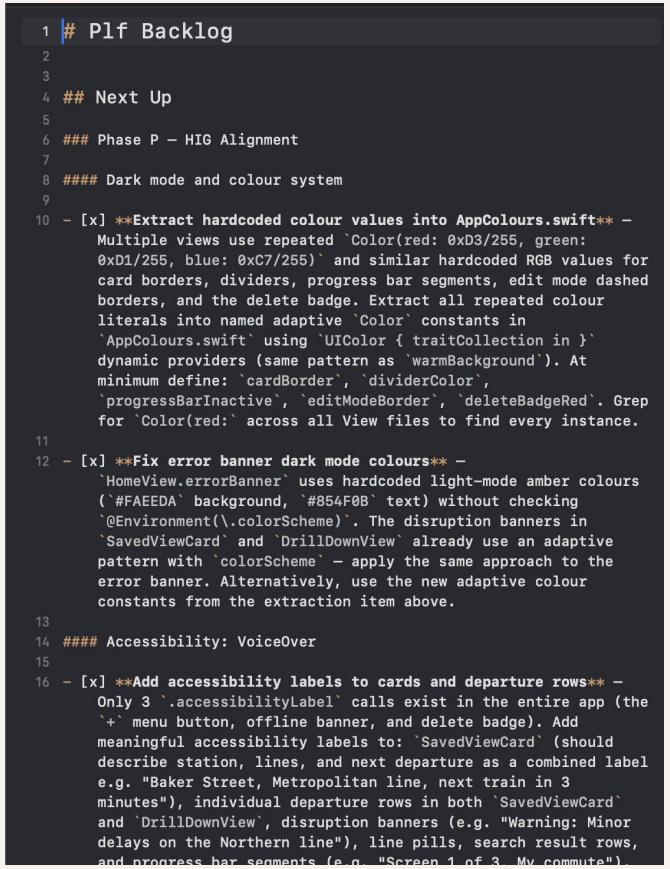


```
# Plf — Apple Human Interface Guidelines Alignment Checklist

This document is a structured audit checklist for evaluating Plf against Apple's Human Interface Guidelines (HIG). It is designed to be fed into Claude Code for autonomous or assisted review of the SwiftUI codebase.

Reference: [Apple Human Interface Guidelines](https://developer.apple.com/design/human-interface-guidelines/)

---

## How to use this file

For each section, Claude Code should:

1. Read the relevant source files listed under **Where to look**
2. Evaluate each checklist item against the current implementation
3. Report findings as **Pass**, **Fail**, or **Needs review** (requires human/device verification)
4. For any Fail, describe the specific violation and suggest a fix
5. Output a summary at the end with pass/fail counts per section

**Important:** This checklist covers principles and patterns, not pixel-level design. Some items (contrast ratios, visual balance, animation feel) require on-device human review — flag these as "Needs review" rather than guessing.

---

## 1. Core Design Principles

Apple's HIG is built on three foundational principles. These are qualitative — Claude Code should flag anything that
```

1. Had Claude Chat draft an audit checklist (markdown file) to assess alignment of my Plf project to Apple's Human Interface Guidelines.

```
# HIG Audit Results — Plf

## HIG Audit Summary — Plf

Date: 2026-04-11
Auditor: Claude Code
Codebase state: fdeccac

| Section                          | Pass | Fail | Needs Review |
|----------------------------------|------|------|--------------|
| 1. Core Design Principles        | 4    | 0    | 7            |
| 2. Navigation                    | 10   | 1    | 2            |
| 3. Layout and Spacing            | 6    | 1    | 5            |
| 4. Typography                    | 5    | 2    | 4            |
| 5. Colour and Dark Mode          | 6    | 4    | 2            |
| 6. Touch Targets and Interactions| 7    | 1    | 4            |
| 7. Lists, Cards, Data Display    | 14   | 1    | 1            |
| 8. Context Menus and Edit Mode   | 10   | 0    | 3            |
| 9. Search                        | 9    | 1    | 0            |
| 10. Alerts and Confirmations     | 9    | 1    | 0            |
| 11. Loading, Error, Empty States | 12   | 2    | 1            |
| 12. Accessibility                | 5    | 6    | 5            |
| 13. SF Symbols and Iconography   | 4    | 1    | 2            |
| 14. Haptics and Feedback         | 3    | 1    | 3            |
| 15. Performance and Responsiveness| 7   | 0    | 2            |
| 16. App Lifecycle and State      | 5    | 1    | 1            |
| 17. App Icon and Launch Screen   | 4    | 1    | 1            |
| **TOTAL**                        | **120** | **23** | **43**     |

### Critical failures (fix before release)
- **Error banner not dark-mode adaptive** — `HomeView.errorBanner` uses hardcoded light-mode amber colours without `@Environment(\.colorScheme)` check (Section 5)
- **ScreenProgressBar inactive segment not adaptive in dark mode** — hardcoded light beige `Color(red: 0xD5..., 0xD1..., 0xCA...)` will be invisible on dark backgrounds (Section 5)
- **Card border/divider colours not adaptive** — multiple hardcoded `Color(red: 0xD3..., 0xD1..., 0xC7...)` values used for card strokes and dividers across `SavedViewCard`, `HomeView`, and `DrillDownView` — these light colours disappear in dark mode (Section 5)
- **Missing accessibility labels on most interactive elements** — only 3 explicit `.accessibilityLabel` in the entire app; cards, departure rows, disruption
```

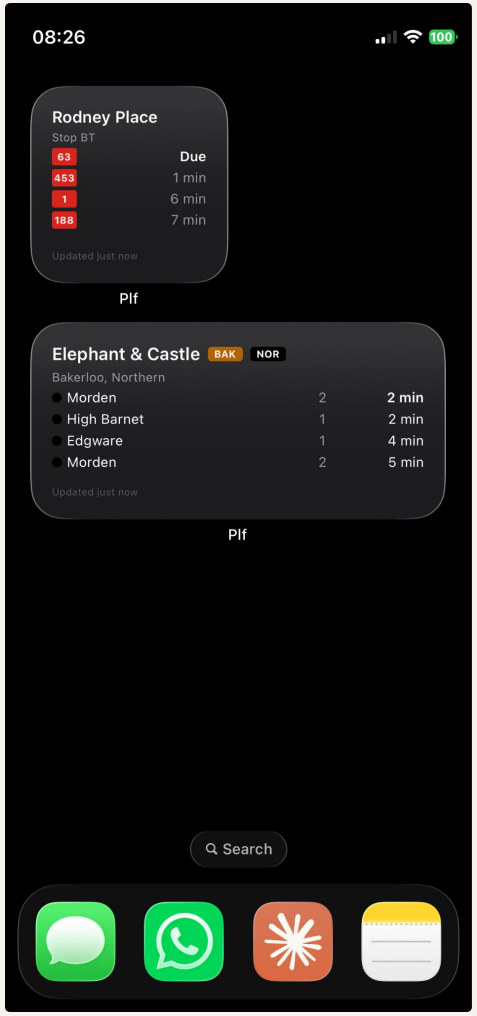
2. Fed it to Claude Code, which outputed audit results.

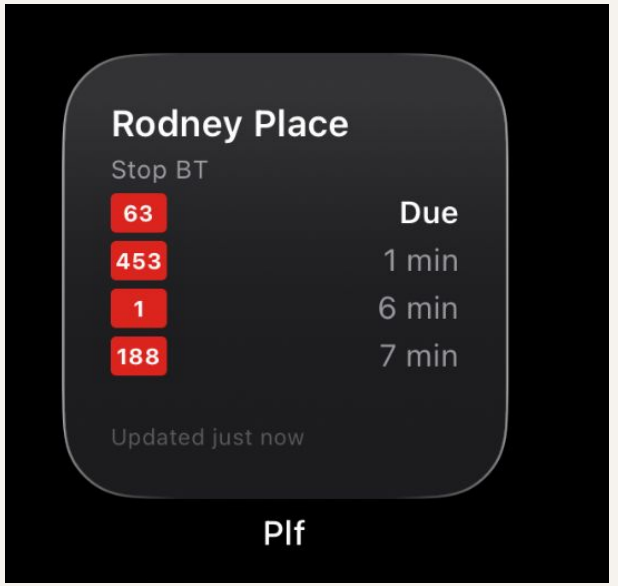


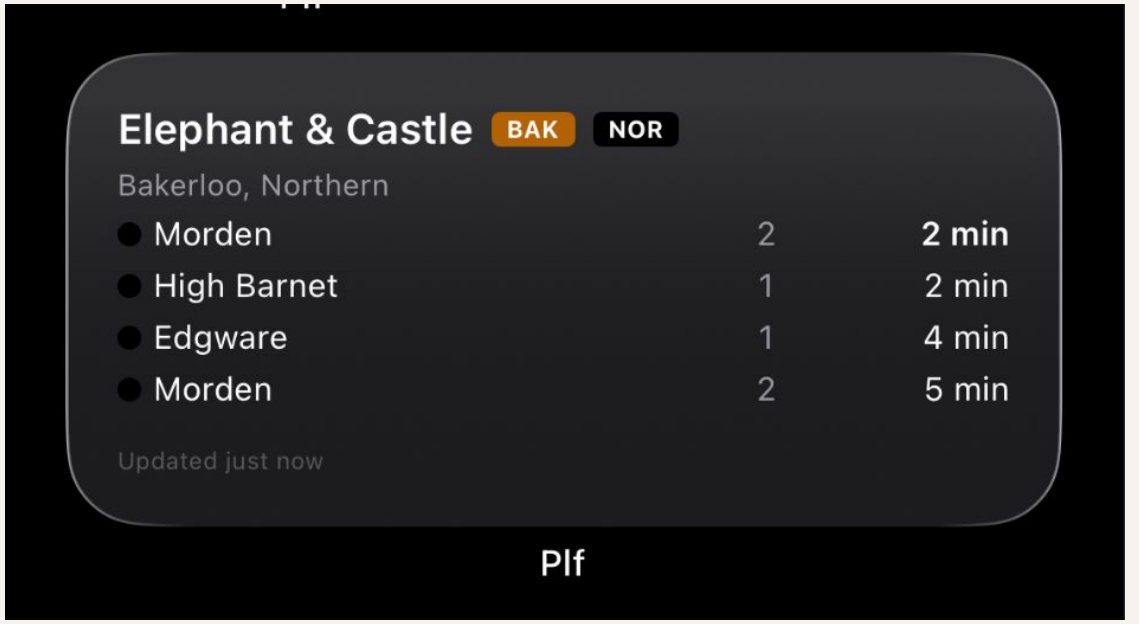
```
# Plf Backlog


## Next Up

### Phase P — HIG Alignment

#### Dark mode and colour system

- [x] **Extract hardcoded colour values into AppColours.swift** — Multiple views use repeated `Color(red: 0xD3/255, green: 0xD1/255, blue: 0xC7/255)` and similar hardcoded RGB values for card borders, dividers, progress bar segments, edit mode dashed borders, and the delete badge. Extract all repeated colour literals into named adaptive `Color` constants in `AppColours.swift` using `UIColor { traitCollection in }` dynamic providers (same pattern as `warmBackground`). At minimum define: `cardBorder`, `dividerColor`, `progressBarInactive`, `editModeBorder`, `deleteBadgeRed`. Grep for `Color(red:` across all View files to find every instance.

- [x] **Fix error banner dark mode colours** — `HomeView.errorBanner` uses hardcoded light-mode amber colours (`#FAEEDA` background, `#854F0B` text) without checking `@Environment(\.colorScheme)`. The disruption banners in `SavedViewCard` and `DrillDownView` already use an adaptive pattern with `colorScheme` — apply the same approach to the error banner. Alternatively, use the new adaptive colour constants from the extraction item above.

#### Accessibility: VoiceOver

- [x] **Add accessibility labels to cards and departure rows** — Only 3 `.accessibilityLabel` calls exist in the entire app (the `+` menu button, offline banner, and delete badge). Add meaningful accessibility labels to: `SavedViewCard` (should describe station, lines, and next departure as a combined label e.g. "Baker Street, Metropolitan line, next train in 3 minutes"), individual departure rows in both `SavedViewCard` and `DrillDownView`, disruption banners (e.g. "Warning: Minor delays on the Northern line"), line pills, search result rows, and progress bar segments (e.g. "Screen 1 of 3. My commute").
```

3. Gave audit results back to Claude Chat to draft new backlog items which I then put in my regular workflow to update the app. Many backlog items were related to accessibility.

# Phase R - Added Widgets



# Phase X & Y - Code Review and Clean Up

***Prompts generated with help from Claude Chat project***

```
Read CLAUDE.md and then perform a read-only codebase audit. Do NOT make an
y changes — only report findings. For each Swift file in plf/plf/ and PlfKi
t/Sources/PlfKit/, analyse and report:

1. **Dead code** — functions, computed properties, enum cases, or type defi
nitions that are never referenced anywhere else in the codebase. Use grep/f
ind to verify before listing.

2. **Large files** — any file over 300 lines. For each, summarise what it d
oes and whether it has a single clear responsibility or has grown to cover
multiple concerns.

3. **Duplicated logic** — patterns, helper functions, or blocks of similar
code that appear in more than one file and could be extracted into a share
d utility.

4. **Inconsistent patterns** — places where similar tasks (error handling,
date formatting, API response processing, view styling) are done differentl
y in different files.

5. **Unused imports** — files that import modules they don't use.

6. **Complexity hotspots** — functions longer than 40 lines or with more th
an 3 levels of nesting.

Format the output as a markdown report grouped by category, with file path
s and line numbers. At the end, rank the top 5 highest-impact improvements
by effort-to-benefit ratio.
```

(1) Read-only code base audit, outputed as a markdown file. Top 5 highest impact improvements by effort-tobenefit ratio.*

```
Read CLAUDE.md, then review the overall architecture of the codebase. Do NO
T make any changes. Report on:

1. **Responsibility boundaries** — Does each file/type have a single clear
job? Flag any types that mix concerns (e.g. a View that also does data fetc
hing, a model that contains display logic, a service that knows about UI st
ate).

2. **Data flow clarity** — Trace how data flows from API call → model → App
State → View for both TfL and Darwin paths. Flag any places where the flow
is indirect, circular, or hard to follow.

3. **PlfKit boundary** — Is the split between the main app target and PlfKi
t clean? Are there types in PlfKit that only the app uses (not the widge
t), or types in the app target that the widget also needs?

4. **Error handling consistency** — How are network errors, decoding error
s, and empty results handled across TfLAPIClient, NationalRailClient, and A
rrivalService? Is there a unified pattern or ad-hoc handling?

5. **State management** — Review AppState. Is it doing too much? Could any
of its responsibilities be extracted? Are there any retain cycles or observ
ation inefficiencies?

6. **Naming audit** — Flag any types, functions, or properties whose names
are misleading, ambiguous, or inconsistent with the naming conventions use
d elsewhere.

Present findings as a prioritised list: what would you fix first if you cou
ld only make 5 changes?
```

(2) Read-only architecture review outputed as a markdown file with top five changes.*

(3) Dropped markdown files into Claude Chat to generate new Backlog items for Phases X & Y.*